EJERCICIO 1

1. Descripción breve del problema

En la cafetería existen dos filas independientes: una para alumnos y otra para trabajadores. La regla de atención prioriza estrictamente a los trabajadores; los alumnos solo son atendidos si la fila de trabajadores se encuentra totalmente vacía. Dada una secuencia de N eventos (llegada de alumnos, llegada de trabajadores o solicitudes de atención), el programa debe procesar los eventos y arrojar los nombres de las personas atendidas en el orden cronológico exacto en el que ocurrió.

2. Tipo de lista elegida y justificación

• Estructura elegida: Cola implementada mediante Lista Enlazada Simple con referencias al frente y al fin.

Justificación: El problema maneja una disciplina de atención FIFO el primero en llegar es el primero en ser atendido para cada colectivo por separado. Al utilizar punteros directos al inicio y al final de la lista, tanto la inserción (encolar) como la extracción se ejecutan en un tiempo constante de O(1). Esto es indispensable ya que el número de eventos puede ascender hasta N = 2 x 105, evitando problemas de desbordamiento de tiempo en la plataforma.

3. Explicación del algoritmo

• Clase Nodo: Contiene un atributo de tipo String para almacenar el nombre de la persona y un puntero siguiente que apunta al próximo nodo en la estructura.

• Clase Cola Cafetería: Administra de manera independiente las operaciones de inserción al final (fin. Siguiente = nuevo) y extracción al frente (frente = frente. Siguiente).

Flujo Principal (Main): Se lee de manera optimizada el número total de eventos N. Mediante un ciclo se identifican las instrucciones:

Si el evento es llega alumnos y trabajadores, se inserta el nombre en su respectiva cola.

```
LLEGA ALUMNO luis
LLEGA TRABAJADOR ana
LLEGA ALUMNO jorge
ATIENDE
ATIENDE
LLEGA TRABAJADOR juan
ATIENDE
```

```
ana
luis
juan
```

- **Veredicto omegaUp:**

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-25 21:59	java	100.00%	Terminada	Correcta	19.71 MB	0.42 s	

- **Enlace a GitHub:**

  Ejercicios_Listas/ejercicio1_FormadosEnLaCafeteria at feature/listas-java · ValeMolina612/Ejercicios_Listas

EJERCICIO 3

1. Descripción breve del problema

Después de un taller, necesitas comprar una botella de agua. Al llegar a la tienda, te encuentras con N marcas diferentes, cada una con un precio distinto. Como requieres conservar dinero para tu pasaje de regreso a casa, el objetivo del programa es identificar y mostrar en la salida el precio de la botella más barata disponible.

2. Tipo de lista elegida y justificación

• Estructura elegida: Lista Simple Enlazada.
• Justificación: Se seleccionó una lista simplemente enlazada para almacenar los precios secuencialmente a medida que son leidos de la entrada estandar. Permite recorrer los nodos de manera lineal para realizar la comparación de los elementos y hallar el valor mínimo con una implementación limpia y directa acorde a los fundamentos de estructuras dinámicas.

3. Explicación del algoritmo

• Clase Nodo: Contiene una variable entera precio para guardar el costo de la marca de agua y una referencia siguiente hacia el proximo nodo.
• Clase ListaSimple; Cuenta con un método de inserción (insertar) para agregar los precios al final de la estructura, y un método de recorrido (ebtene Minimo) que inicializa una variable con el primer elemento y la compara contra el resto de los nodos para aislar el menor costo.
• Flujo Principal (Main): Lee la cantidad total de marcas N, procesa la linea con los N precios utilizando StringTokenizer, los almacena en la lista enlazada y tinalmente imprime el resultado minimo obtenido.

4. Pruebas realizadas

• Caso Normal 1: Entrada con 5 precios (9 10 8 7 11). Salida esperada: 7.
• Caso Normal 2: Entrada con 3 precios (20 12 16). Salida esperada: 12.
• Caso Límite: Prueba con N = 1 para asegurar que la lista con un solo elemento procese y devuelva el precio correctamente sin errores.



- **Veredicto omegaUp:**

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida
2026-10-25 23:08	java	100.00%	Terminada	Correcta

- **Enlace a GitHub:**

Ejercicios_Listas/ejercicio3_ComprandoAgua at feature/listas-java · ValeMolina612/Ejercicios_Listas

EJERCICIO 5

Descripción breve del problema: El programa recibe una cantidad N de productos, cada uno con su respectivo nombre y precio. Debe organizarlos en un formato de ticket de mayor a menor precio, enumerados del 1 al N separados por el caracter " | ", y finalmente mostrar la sumatoria total de los precios.

Tipo de lista elegida y justificación: Se seleccionó una **Lista Enlazada Simple**, ya que permite realizar inserciones de forma ordenada de manera dinámica sin necesidad de redimensionar arreglos estáticos, adaptándose perfectamente al límite de hasta 100 elementos indicado por el problema.

Explicación del algoritmo: Se implementó un método de inserción ordenada (insertarOrdenado) que evalúa el precio del nuevo producto frente a la cabeza y los nodos siguientes mediante punteros (siguiente), ubicándolo en el lugar correcto de manera descendente. Asimismo, se acumula de forma paralela la suma total de los precios.

Pruebas realizadas:

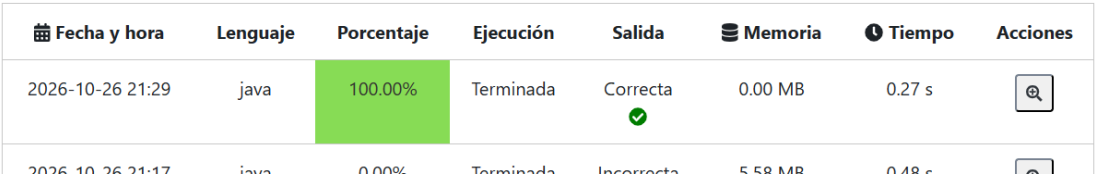
- **Caso 1 (Normal):** Con los datos provistos en el ejemplo de omegaUp (8 productos).
- **Caso 2 (Límite):** Con un solo elemento ($N = 1$).
- **Caso 3 (Grande):** Con múltiples productos (por ejemplo, 10 o más).

```
8
axion 23.60
fud 23.00
LaLa 23.01
Petalo 129.5
2Kg_manzanas 58.6
BimboG 30.5
MayonesaM 50.5
Colgate 23
```

```
1 | Petalo | 129.5
2 | 2Kg_manzanas | 58.6
3 | MayonesaM | 50.5
4 | BimboG | 30.5
5 | axion | 23.6
6 | LaLa | 23.01
7 | Colgate | 23
8 | fud | 23

361.71
```

- **Veredicto omegaUp:**



Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-26 21:29	java	100.00%	Terminada	Correcta	0.00 MB	0.27 s	
2026-10-26 21:17	java	0.00%	Terminada	Incorrecta	5.58 MB	0.48 s	

- **Enlace a GitHub:**

[Ejercicios_Listas/ejercicio5_Supermercado at feature/listas-java · ValeMolina612/Ejercicios_Listas]

Conclusiones: Dificultades encontradas como el manejo correcto de los punteros al insertar al inicio o en medio de la lista y cómo se resolvieron mediante la verificación de condiciones en las estructuras if-else y los ciclos while

EJERCICIO 7

. Descripción breve del problema

El problema consiste en procesar los nombres y promedios de N alumnos. El programa debe aplicar un filtro para seleccionar únicamente a aquellas personas que hayan obtenido un promedio académico de **9.5 o más**. Finalmente, los nombres de los alumnos que califiquen deben imprimirse ordenados estrictamente de forma alfabética (de la A a la Z).

2. Tipo de lista elegida y justificación

- Tipo de lista utilizada: Lista Enlazada Simple.
- Justificación: Se seleccionó una lista enlazada simple porque permite realizar inserciones de manera dinámica y ordenada (ascendente por orden alfabético) conforme se van evaluando los datos de entrada, sin necesidad de declarar arreglos de tamaño estático o realizar complejos reordenamientos posteriores.

3. Explicación del algoritmo

- Estructura de Nodo (NodoEstudiante): Almacena el nombre del estudiante (String), su promedio (double) y un puntero siguiente que apunta al próximo nodo en la memoria.
- Inserción Ordenada (ListaEstudiantesHonor): El método evalúa cada nuevo nombre frente a la cabeza de la lista utilizando el método compareToIgnoreCase(). Si la lista está vacía o el nuevo nombre precede alfabéticamente a la cabeza, se inserta al inicio. De lo contrario, se recorre la lista mediante el desplazamiento del puntero (actual = actual.siguiente) hasta encontrar la posición exacta de inserción (en medio o al final).
- Filtro y Salida: El método main lee los datos estructurados en bloques de dos líneas por alumno (nombre y promedio), descarta a los menores de 9.5, inserta a los seleccionados mediante la lista enlazada y finalmente imprime el cuadro de honor ordenado.

4. Pruebas realizadas

- Caso de prueba 1: Conjunto estándar de alumnos con promedios variados (por ejemplo, los 5 casos del ejemplo de omegaUp).
- Caso de prueba 2: Alumnos con promedios exactamente en el límite (9.5) y valores inferiores para comprobar la efectividad del filtro.
- Caso de prueba 3 : Listas amplias con múltiples estudiantes para evaluar la estabilidad del ordenamiento alfabético.
- Veredicto de omegaUp: [Insertar aquí captura de pantalla del veredicto Accepted / 100%]

5. Enlace al código en GitHub

- **Repositorio:** [Ejercicios_Listas/ejercicio7_CuadrodeHonor at feature/listas-java · ValeMolina612/Ejercicios_Listas]

6. Conclusiones

- Dificultades encontradas: La principal dificultad inicial consistió en asegurar la lectura correcta de nombres compuestos con espacios en blanco utilizando el flujo estándar, así como garantizar que el ordenamiento alfabético ignorara las diferencias entre mayúsculas y minúsculas.

```
5
Pepito Perez
9.6
Julian Cardenas
10.0
Felix Garduza
9.9
Martin Gonzalez
9.2
Jose Soto
8.5
```

statement_001.out

```
Felix Garduza
Julian Cardenas
Pepito Perez
```

Fecha y hora	Lenguaje	Porcentaje	Ejecución	Salida	Memoria	Tiempo	Acciones
2026-10-26 20:29	java	100.00%	Terminada	Correcta	9.71 MB	2.05 s	